COMP 333
Fall 2026

Program Properties and Types (Answers)

We often want our programs to have certain properties. Some common properties of interest are below:

- Only certain values are permitted to be passed / assigned to certain variables
- We do not access arrays out of bounds
- We do not call functions or methods that don't exist
- We pass the correct number of parameters to any functions or methods
- We do not attempt to dereference `null`

The specific properties of interest for a given language are determined by the language's design.

In a **strongly-typed** language, these properties are automatically checked for us, and some fixed behavior occurs if these properties don't hold. Exactly what this behavior is depends on the language's design. These properties could be checked at compile-time, runtime, or some combination thereof.

In a **weakly-typed** language, these properties are **not** always automatically checked for us. The language nonetheless assumes these properties hold. If the language doesn't automatically ensure a given property holds, then it's up to programmers to ensure these hold, e.g., by performing some check at runtime. If the programmer fails to ensure these checks hold, it usually means the program will have **undefined behavior** - there is no fixed behavior when a property fails to hold, and anything could happen.

A common way to help enforce that properties hold is via types. In a **statically-typed** language, variables have types which are fixed at compile-time, and cannot change. The consistency of these types is checked at compile-time. If an inconsistency is detected, the program fails to compile with a **type error**. Various properties can be encoded and enforced with types. Because this checking is done at compile-time, such checks are generally considered "free": they don't incur any runtime cost.

In a **dynamically-typed** language, this checking of types is performed at runtime. Variables do not have fixed types, and can contain values of different types at different times. Because the check is delayed until runtime, there is a runtime cost for performing these checks. However, this is more flexible than static typing, because variables do not need fixed types.

Strong/weak typing are opposites, as are static/dynamic typing. However, a language can be both strong and static (e.g., Java), strong and dynamic (e.g., JavaScript), or weak and static (e.g., C/C++). The combination of weak and dynamic is theoretically

possible, but not very useful - this means that certain properties must hold, and some might be checked at runtime, but likely aren't.

Assembly languages are generally **untyped**, meaning there are no inherent types at all. Assembly instructions generally operate over series of bits, and perform some sort of transformation over these bits. While there is an implicit assumption that these bits do encode something meaningful, the processor has no inherent understanding of what these bits encode, and it's entirely up to us to ensure that we use the right instructions at the right time. (I can see an argument that the combination of weak and dynamic is not that different from untyped, and I wouldn't disagree.)

Languages which are strong and static often have both compile-time and runtime checks. For example, in Java, consider this array access: `someArr[index]` Java uses static types to ensure that `someArr` is an array, and that `index` is an `int`; these checks are done at compile-time. However, Java delays the check that `index` is in bounds of `someArr` until runtime. Most strongly-typed statically-typed languages delay the bounds check until runtime, because it is difficult to perform this check at compile-time. Conversely, for a weakly-typed language, the compiler would generally check that `someArr` is an array, and `index` is an `int` at compile-time via types; C does exactly this. However, C will merely *assume* that `index` is in bounds at runtime, without performing a check; hence, C is weakly-typed. It's up to the programmer to ensure that `index` is in bounds. In C, it's undefined behavior to access an array out of bounds.

Type-inferred languages are statically-typed, but the programmer does not need to manually annotate at least some variables with their types. Instead, types are determined from context, like so (in Scala, a type-inferred language): `val x = "a"` In this case, because `"a"` is a `String`, and we initialize `x` with `"a"`, the Scala compiler knows that `x` must have type `String`. This determination is performed **at compile time**, so this behaves just the same as a regular statically-typed language, even though it may initially look as if it's dynamically-typed.

----Begin relevant exercises----

1.) Consider the following code snippet in some unknown language:

```
boolean x = hopefullyArray[unknownPosition];
```

The following checks are relevant:

1. Ensure that `hopefullyArray` is an array
2. Ensure that `unknownPosition` is an integer
3. Ensure that `unknownPosition` is in bounds of the array
4. Ensure that `hopefullyArray` is an array of `boolean` values

Write the numbers of the checks which are most likely to be performed when (runtime or compile-time), depending on what kind of language this program is written in.

	Statically-Typed	**Dynamically-Typed**
Strongly-Typed	Runtime: 3 Compile-time: 1, 2, 4	Runtime: 1, 2, 3, 4 Compile-time:
Weakly-Typed	Runtime: Compile-time: 1, 2, 4	

2.) Consider the following code snippet in some unknown object-oriented language:

```
int y = foo.bar(1, true);
```

The following checks are relevant:

1. Ensure that `foo` is an object
2. Ensure that `foo` is not `null`
3. Ensure that `foo` has a `bar` method
4. Ensure that the `bar` method takes two parameters
5. Ensure that `bar`'s first parameter should be an integer
6. Ensure that `bar`'s second parameter should be a boolean
7. Ensure that `bar` returns an `int`

Identify which checks are most likely to be performed when for each kind of language.

	Statically-Typed	**Dynamically-Typed**
Strongly-Typed	Runtime: 2 Compile-time: 1, 3, 4, 5, 6, 7	Runtime: 1, 2, 3, 4, 5, 6, 7 Compile-time:
Weakly-Typed	Runtime: Compile-time: 1, 3, 4, 5, 6, 7	

3.) Consider the following code snippet, written in some unknown language:

```
define_function myFunc(x, y) {
  return x + y;
}

a = 1
b = 2
myFunc(a, b)
```

3.a.) In 1-2 sentences, provide an explanation why this could be a dynamically-typed language.

No types have been annotated.

3.b.) In 1-2 sentences, provide an explanation why this could be a statically-typed language. As a hint, there is a special kind of statically-typed language mentioned in the above text, which may be relevant.

Variables never change their types, so this could be type-inferred.

3.c.) Write out at least one desirable property which a program in this language should have, based on what the above code does. As a hint, what sort of checks would you need to perform to ensure the above code makes sense?

Possibilities (among others):

- `a` is in scope
- `a` has type `int`
- `b` is in scope
- `b` has type `int`
- `x` is in scope
- `x` has type `int`
- `y` is in scope
- `y` has type `int`
- function `myFunc` exists
- `myFunc` takes two parameters
- `myFunc`'s first parameter is an `int`
- `myFunc`'s second parameter is an `int`
- `myFunc` returns an `int`